

13

Cyber Security
Codec Technologies India
Internship 2026

Ransomware Behaviour Detection

Behaviour detector, not a simulator

AT A GLANCE

140 / 50

Alert score

5

Indicators fired

83

FS events

41

High-entropy files

0

Files harmed outside sandbox

17 / 17

Tests re-run

True

Originals preserved

VERDICT

TESTS 17/17

RE-RUN HERE

DECLARED SUBSTITUTION

NO ENCRYPTION

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

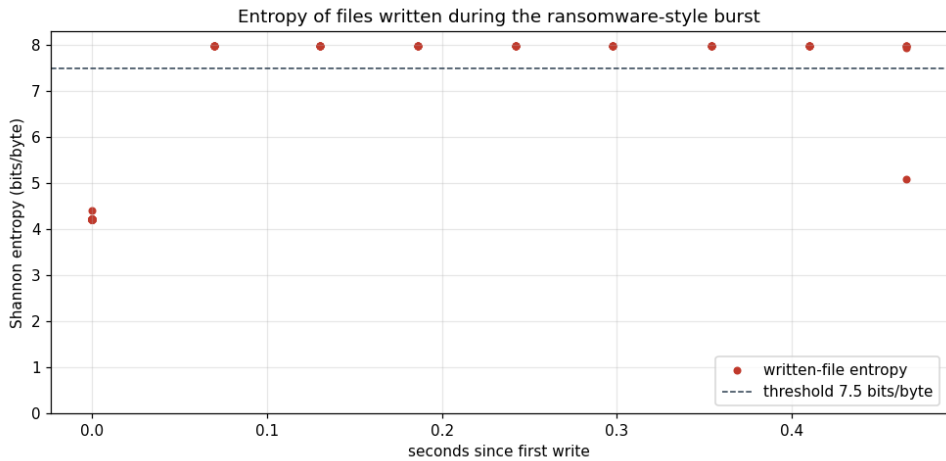
Objective & brief mapping

The brief asked for a "ransomware simulation". Writing anything that encrypts, renames or destroys real files is dangerous and unethical, so **for safety I built the defence**: a monitor that recognises the behaviour real ransomware exhibits. To have something to detect without building malware, I include a benign, sandboxed workload that imitates the observable side-effects only — there is no real cryptographic locking anywhere in this project.

Architecture

The benign workload runs inside one disposable /tmp sandbox, writing new random-byte files as a stand-in for ciphertext output and never touching originals. A polling snapshot-diff watcher feeds Events to the detector, which sums weighted indicators and, past threshold 50, performs a simulated (logged, not executed) response and quarantines a sandbox copy.

workload (benign, /tmp sandbox) -> FS changes
watcher (poll snapshot-diff) -> Event(kind,path,ts,entropy)
monitor: sliding-window indicators -> score>=50 -> ALERT + sim response



Per-file entropy vs the 7.5 bits/byte threshold (results/entropy_timeline.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # attack + control scenarios (~3 s)
python3 unittest-style runner over tests/ # 17 tests
```

The artefacts each figure and table above are drawn from:

- **detection_log.json** — full detector state + safety witness
- **indicators.json** — every indicator + weight
- **entropy_timeline.png** — per-file entropy vs threshold
- **alert_report.md / response.json** — incident + simulated response

Method & implementation

Six weighted indicators: canary_tripwire (+40), mass_modification_rate (+30), entropy_spike (+30, Shannon >= 7.5 bits/byte), ransom_note (+25), mass_rename_burst (+20), suspicious_extension (+15). Every write path is asserted inside the sandbox first, so the code cannot touch anything outside it even if a path were built wrongly.

Results

From results/detection_log.json and indicators.json. Attack scenario alerted at 140/50; control scenario stayed quiet at 0/50 (correct).

Indicator / metric	Value
Alert score	140 / threshold 50 -> ALERT
Indicators fired	canary, mass-mod, entropy, note, extension (5)
FS events / high-entropy files	83 / 41
Originals preserved	True
Control scenario	0 / 50 -> no alert (correct)
Files harmed outside sandbox	0

Detection indicators (threshold 50)

Indicator	Weight	Catches
canary_tripwire	+40	honeyfile touched
mass_modification_rate	+30	write burst
entropy_spike	+30	ciphertext-like output
ransom_note / extension	+25 / +15	note + .locked

13

Cyber Security
Codec Technologies India
Internship 2026

Ransomware Behaviour Detection

Behaviour detector, not a simulator

AT A GLANCE

140 / 50

Alert score

5

Indicators fired

83

FS events

41

High-entropy files

0

Files harmed outside sandbox

17 / 17

Tests re-run

True

Originals preserved

VERDICT

TESTS 17/17

RE-RUN HERE

DECLARED SUBSTITUTION

NO ENCRYPTION

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Verification

Re-ran here, 17/17 (~0.5 s) — the largest suite in the repo. The key test asserts that a witness file outside the sandbox is byte- and mtime-identical after a full workload run.

Test group	Checks	Result
Entropy	random >7.9, text <5.0, constant 0.0	PASS
Indicators	canary / mass-mod / entropy / extension fire	PASS
End-to-end	detector catches suspect activity	PASS
Safety	nothing outside sandbox modified; /etc/passwd refused	PASS
Total	17 tests	17/17

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Polling snapshot-diff stands in for OS file-system events; the response is simulated, never executed.

Ran here	Real production	Note
polling watcher	watchdog / fanotify / ETW	kernel-level in production
logged SIGSTOP/SIGKILL	real process kill	simulated in this lab
benign /tmp workload	actual ransomware	never built (ethics)

Ethics, data integrity & declared substitutions

Defensive and educational. **Declared substitution (the core ethical decision, copied from my README):** the requested ransomware simulation is delivered as a **behaviour detector** — it contains no ransomware and no encryption of user data. The only "malicious" activity is a benign stand-in confined to a project-created /tmp sandbox that writes random bytes to new files, preserves all originals, and is torn down. The process-killing response is simulated and logged, not executed against the host.

Learnings

Reframing the brief from "build the weapon" to "build the defence" is the professional judgement I am most proud of here. Keying detection on the file-system fingerprint (rate + entropy + canary) rather than on any single signature is what lets one detector generalise across ransomware families it has never seen.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The workload is a behavioural stand-in; it reproduces the file-system fingerprint only.
- Thresholds are tuned for the demo and need calibration against a real baseline.
- Polling has latency between snapshots; production uses OS file-system events.
- Entropy is sampled from the first 64 KB of each file for speed.
- Six weighted indicators; alert threshold 50, attack scenario scored 140.
- Every write path is asserted inside the /tmp sandbox before writing.
- Originals are preserved next to their .locked twins; files harmed outside sandbox = 0.
- Control scenario (slow legitimate writes) correctly scored 0/50.
- Attack scenario: 83 FS events, score 140/50, 5 indicators fired.
- Planted 40 decoy docs + 1 honeyfile; 40 .locked stand-ins written.
- 41 newly-written files measured as high-entropy (>= 7.5 bits/byte).
- Control scenario scored 0/50 (no false alarm).
- Safety witness: files modified outside sandbox = 0.
- Note scanner found 1 ransom-note file.
- assert_inside_sandbox refuses an out-of-sandbox path like /etc/passwd.
- References: MITRE T1486, CISA StopRansomware, NIST IR 8374.
- Reference: MITRE ATT&CK; T1486 (Data Encrypted for Impact).